① → Once the I/P size n becomes large enough, merge sort, with its $\Theta(n \lg n)$ worst case running time, beats insertion sort, whose worst-case running time is $\Theta(n^2)$.

→ Although we can determine the exact running time of an algorithm, the extra precision is usually not worth the effort. For large enough inputs, the multiplicative constants and the lower order terms are dominated by the effects of the I/P size itself.

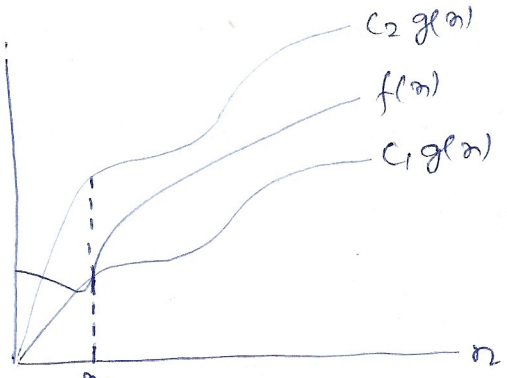
→ When we look at I/P sizes large enough to make only the order of growth of the running time relevant, we are studying the asymptotic efficiency of algorithms. That is, we are concerned with how the running time of an algorithm increases with the size of the I/P in the limit, as the size of the I/P increases without bound. Usually, an algorithm that is asymptotically more efficient will be the best choice for all but very small inputs.

② Θ notation (theta notation):

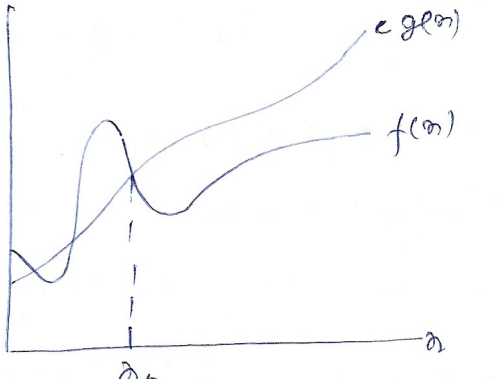
→ For an given function $g(n)$, $\Theta(g(n))$ is the set of functions

$$\Theta(g(n)) = \{ f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ such that } 0 \le c_1 g(n) \le f(n) \le c_2 g(n) \text{ for all } n \ge n_0 \}$$

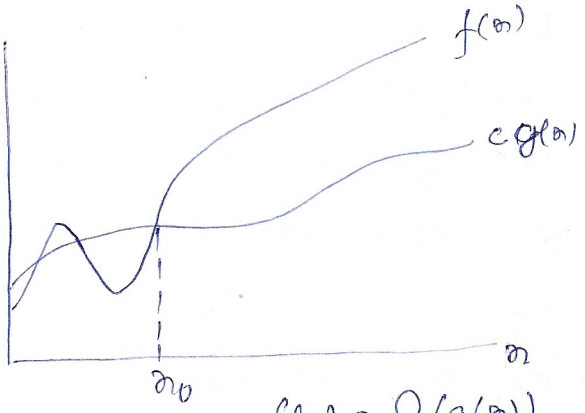
→ A function $f(n)$ is said to belong to the set $\Theta(g(n))$ if there exist positive constants c_1, c_2 ~~and n_0~~ such that $f(n)$ can be sandwitched between $c_1 g(n)$ and $c_2 g(n)$ for sufficiently large n. We write $f(n) \in \Theta(g(n))$ or $f(n) = \Theta(g(n))$ in that case.



$f(n) = \Theta(g(n))$



$f(n) = O(g(n))$



$f(n) = \Omega(g(n))$

→ For all $n \geq n_0$, the function $f(n)$ is equal to $g(n)$ within a constant factor. We say that $g(n)$ is an asymptotically tight bound for $f(n)$.

(3) O-notation (pronounced "big-oh")

The O-notation provides asymptotic upper bound to a function.

$O(g(n)) = \{ f(n) :$ there exist positive constants c and n_0 such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0 \}$

→ We write $f(n) = O(g(n))$ to indicate that $f(n)$ is a member of the set $O(g(n))$

→ $f(n) = \Theta(g(n))$ implies that $f(n) = O(g(n))$, since Θ is a stronger notation than O.

→ When we write $f(n) = O(g(n))$, we are merely claiming that some constant multiple of $g(n)$ is an asymptotic upper bound on $f(n)$, without any claim about how tight an upper bound it is. Thus, we may write $n = O(n^2)$. The O-notation may thus stand for both tight and loose

upper bound.

→ Since O-notation describes an upper bound, when we use it to bound the worst-case running time of an algorithm, we have a bound on the running time of the algorithm on every I/P.

→ When we say that the running time of insertion sort is $O(n^2)$ we mean that the worst-case running time is $O(n^2)$.

④ Ω notation (pronounced "big-omega")

Ω-notation provides an asymptotic lower bound.

$\Omega(g(n)) = \{ f(n)$: there exist positive constants c and n_0 such that $0 \leq cg(n) \leq f(n)$ for all $n \geq n_0 \}$

⑤ Theorem

$f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$

→ Since Ω-notation describes a lower bound, when we use it to bound the best case running time of an algorithm, by implication we also bound the running time of the algorithm on arbitrary inputs. For eg, the best case running time of insertion sort is $\Omega(n)$, which implies that the running time of insertion sort is $\Omega(n)$. The running time of insertion sort therefore falls between $\Omega(n)$ and $O(n^2)$.

⑥ o-notation (pronounced "little-oh")

We use the o-notation to denote an upper bound that is not asymptotically tight.

$o(g(n)) = \{ f(n)$: for "any" positive constant $c > 0$ there exists a constant $n_0 > 0$ such that $0 \leq f(n) < cg(n)$ for all $n \geq n_0 \}$

For example, $2n = o(n^2)$ but $2n^2 \neq o(n^2)$.

⇒ The definitions of O-notation and o-notation are similar. The main difference is that in $f(n) = O(g(n))$, the bound $0 \le f(n) \le c\,g(n)$ holds for "some" constant $c > 0$, but in $f(n) = o(g(n))$ the bound $0 \le f(n) < c\,g(n)$ holds for all constants $c > 0$.

Thus, in case of o-notation we have:-

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 0$$

(7) ω-notation (pronounced "little-omega")

$\omega(g(n)) = \{ f(n)$: for "every" positive constant $c > 0$, there exists a constant $n_0 > 0$ such that $0 \le c\,g(n) < f(n)$ for all $n \ge n_0 \}$

or, $f(n) \in \omega(g(n))$ if and only if $g(n) \in o(f(n))$

for eg, $n^2/2 = \omega(n)$, but $n^2/2 \ne \omega(n^2)$.

The relation $f(n) = \omega(g(n))$ implies that:

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = \infty$$

(8) Comparison of functions

Transitivity

$f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$ imply $f(n) = \Theta(h(n))$

$f(n) = O(g(n))$ and $g(n) = O(h(n))$ imply $f(n) = O(h(n))$

$f(n) = \Omega(g(n))$ and $g(n) = \Omega(h(n))$ imply $f(n) = \Omega(h(n))$

$f(n) = o(g(n))$ and $g(n) = o(h(n))$ imply $f(n) = o(h(n))$

$f(n) = \omega(g(n))$ and $g(n) = \omega(h(n))$ imply $f(n) = \omega(h(n))$

Reflexivity

$f(n) = \Theta(f(n))$

$f(n) = O(f(n))$

$f(n) = \Omega(f(n))$

Symmetry

$f(n) = \Theta(g(n))$ if and only if $g(n) = \Theta(f(n))$

Transpose symmetry

$f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$

$f(n) = o(g(n))$ if and only if $g(n) = \omega(f(n))$

→ We say that $f(n)$ is asymptotically smaller than $g(n)$ if $f(n) = o(g(n))$, and $f(n)$ is asymptotically larger than $g(n)$ if $f(n) = \omega(g(n))$.

→ Not all functions are asymptotically comparable.

(9) Standard notations and common functions

→ Monotonicity

$f(n)$ is said to be monotonically increasing if :-

$$m \leq n \Rightarrow f(m) \leq f(n)$$

$f(n)$ is said to be monotonically decreasing if :-

$$m \leq n \Rightarrow f(m) \geq f(n)$$

$f(n)$ is strictly increasing if :-

$$m \leq n \Rightarrow f(m) < f(n)$$

$f(n)$ is strictly decreasing if :-

$$m < n \Rightarrow f(m) > f(n)$$

→ Floors and ceilings

$x - 1 < \lfloor x \rfloor \leq x \leq \lceil x \rceil < x + 1$

If n is an integer, we have :-

$\lceil n/2 \rceil + \lfloor n/2 \rfloor = n$

For any real no. $n \geq 0$ and integers $a, b > 0$:-

$\lceil \lceil n/a \rceil / b \rceil = \lceil n/ab \rceil$

$\lfloor \lfloor n/a \rfloor / b \rfloor = \lfloor n/ab \rfloor$

$\lceil a/b \rceil \leq (a + (b-1))/b$

$\lfloor a/b \rfloor \geq (a - (b-1))/b$

The floor and ceiling functions are both monotonically increasing.

→ Modular arithmetic

$a \bmod n = a - \lfloor a/n \rfloor n$

If $a \bmod n = b \bmod n$, we write $a \equiv b \pmod n$ and say that a is equivalent to b, modulo n. This is possible if n is a divisor of $b - a$.

(5) Polynomials

a polynomial in n of degree d :-

$$p(n) = \sum_{i=0}^{d} a_i n^i \qquad \left[= \Theta(n^d) \right]$$

Exponentials

For all n and $a \geq 1$, the function a^n is monotonically increasing in n. When convenient we assume $0^0 = 1$.

→ The rates of growth of polynomials and exponentials can be related by the following fact :-

For all real constants a and b such that $a > 1$:-

$$\lim_{n \to \infty} \frac{n^b}{a^n} = 0$$

(n^b → polynomial func., a^n → exponential func.)

from which we conclude that :

$$n^b = o(a^n)$$

"Any exponential function with a base strictly greater than 1 grows faster than any polynomial function."

→ $$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots$$

Thus, $e^x \geq 1 + x$

When $|x| \leq 1$, we have

$$1 + x \leq e^x \leq 1 + x + x^2$$

→ For all x :-

$$\lim_{n \to \infty} \left(1 + \frac{x}{n}\right)^n = e^x$$

(11) $\lg n = \log_2 n$

$\ln n = \log_e n$

$\lg^k n = (\lg n)^k$

$\lg \lg n = \lg(\lg n)$

If we hold $b > 1$ constant, then for $n > 0$, $\log_b n$ is strictly increasing.

→ $$a = b^{\log_b a}$$

$$\log_b a = \frac{\log_c a}{\log_c b}$$

$$a^{\log_b c} = c^{\log_b a}$$

⇨ We say that a function $f(n)$ is polylogarithmically bounded if $f(n) = O(\lg^k n)$ for some constant k.

⇨ We can relate the growth of polynomials and polylogarithms by substituting $\lg n$ for n and 2^a for a in the equation $\lim_{n \to \infty} \frac{n^b}{a^n} = 0$, yielding :-

$$\lim_{n \to \infty} \frac{\lg^b n}{(2^a)^{\lg n}} = \lim_{n \to \infty} \frac{\lg^b n}{n^a} = 0$$

i.e., $\lg^b n = o(n^a)$ for any constant $a > 0$.

"Any positive polynomial function grows faster than any polylogarithmic function".

[exponential > (grows faster than) polynomial > (grows faster than) polylogarithmic]

(12) Factorials

$$n! = o(n^n)$$
$$n! = \omega(2^n)$$
$$\lg(n!) = \Theta(n \lg n)$$

Functional iteration

$$f^{(i)}(n) = \begin{cases} n & \text{if } i = 0 \\ f(f^{(i-1)}(n)) & \text{if } i > 0 \end{cases}$$

(13) Iterated logarithm function

$\lg^* n$ (read "log star of n") is the no. of times the logarithm should be applied to n to let it be ≤ 1.

eg: $\lg^* 2 = 1$, $\lg^* 4 = 2$, $\lg^* 16 = 3$, $\lg^* 65536 = 4$, $\lg^*(2^{65536}) = 5$